

Mid-Session Requirements: Transcript Enhancement

Overview

This mid-session enhancement adds **transcript functionality** to your audio customer support agent. You'll modify the pipeline to return both audio responses AND text transcripts, making the system more accessible and debuggable.

New Requirements

Core Enhancement

Your system must now return **both audio and transcript** for every audio interaction:

```
{
  "success": true,
  "audio_response": "base64_encoded_audio_data",
  "transcript": {
    "user_input": "What is your return policy?",
    "agent_response": "We offer a 30-day return policy for all products..."
  },
  "processing_time_ms": 2050
}
```

Current Server vs Enhanced Requirements

Current Implementation

Text Endpoint (/chat/text):

```
class TextResponse(BaseModel):
    response_text: str
    audio_available: bool
    processing_time_ms: int # Only basic timing
```

Audio Endpoint (/chat/audio):

- Returns raw audio bytes (Content-Type: audio/mpeg)
- No transcript or metadata included
- No JSON response format

What You Need to Add

Implementation Requirements

1. Pipeline Modifications (Required)

File: src/pipeline.py

You must modify your pipeline to capture and return transcripts at each stage:

```
class TranscriptData:
    """Data structure for transcript information"""
    user_input: str
    agent_response: str

    async def process_audio_with_transcript(self, audio_bytes: bytes, **kwargs) ->
    Tuple[bytes, TranscriptData, int]:
        """
        TODO: Implement enhanced audio processing that returns audio, transcript,
        and timing.

        Returns:
            Tuple[bytes, TranscriptData, int]: (response_audio, transcript_data,
            processing_time_ms)
        """
```

2. API Endpoint Updates (Required)

File: `src/api/server.py`

Modify existing endpoints to include transcript data:

Update `/chat/audio` endpoint:

- Change from returning raw audio bytes to JSON response
- Include base64-encoded audio + transcript + processing time

Update `/chat/text` endpoint:

- Keep existing `TextResponse` structure (no changes needed)
- Current response already includes `processing_time_ms`

```
# Modified audio endpoint should return:
{
  "success": true,
  "audio_response": "base64_encoded_audio_data",
  "transcript": {
    "user_input": "transcribed_text",
    "agent_response": "agent_response_text"
  },
  "processing_time_ms": 2050
}
```

3. Streamlit UI Enhancement (Required)

File: `streamlit_app.py`

Modify the existing Audio Chat tab to display transcripts:

Required Changes:

1. **Parse JSON Response:** Handle the new JSON format from `/chat/audio` endpoint
2. **Extract Audio Data:** Decode base64 audio for playback
3. **Display Transcript:** Show user input and agent response text alongside audio controls
4. **Show Processing Time:** Display the processing time metrics

UI Layout:

- Left side: Audio controls (record, play response)

- Right side: Transcript display (user question + agent response)
- Bottom: Processing time information

Expected Output Format

Audio Response Example

```
{
  "success": true,
  "audio_response":
  "UklGRigAAABXQVZFZm10IBAAAAABAAEAQB8AAEAfAAABAAgAZGF0YQAAAAA=",
  "transcript": {
    "user_input": "How long does shipping take?",
    "agent_response": "Shipping typically takes 3-5 business days for standard
delivery within the US. Express shipping (1-2 days) is available for an
additional $15 fee. All orders placed before 2 PM EST ship the same day.",
    "processing_metadata": {
      "stt_confidence": 0.92,
      "response_generation_time_ms": 1100,
      "tts_generation_time_ms": 650,
      "total_processing_time_ms": 1750
    }
  }
}
```

Text Response Example

```
{
  "success": true,
  "response_text": "We offer a 30-day return policy for all products purchased
from our store...",
  "processing_metadata": {
    "response_generation_time_ms": 950,
    "rag_documents_retrieved": 3,
    "rag_search_time_ms": 200
  }
}
```

UI Enhancement Requirements

Enhanced Chat Tab Features

- 1. **Dual Display:** Audio controls on left, transcript on right
- 2. **Real-time Updates:** Show processing as it completes
- 3. **Processing Time Display:** Show total response time

UI Mockup Structure

Enhanced Audio Chat	
Audio Controls	Transcript
[Record]	User: "What is your return
[Stop]	policy?"
[Play Response]	
	Agent: "We offer a 30-day
Processing Time:	return policy..."
• Total: 2.05s	
	Processing: 2.05s

Technical Implementation Details

Pipeline Changes

Your `AudioSupportPipeline` class needs these new methods:

```
class AudioSupportPipeline:
    async def process_audio_with_transcript(self, audio_bytes: bytes, **kwargs)
-> Tuple[bytes, Dict, int]:
        """Process audio and capture transcript data"""

    async def process_text_with_timing(self, text: str, **kwargs) -> Tuple[str,
int]:
        """Process text and capture processing time"""

    def _create_transcript_data(self, user_input: str, agent_response: str) ->
Dict:
        """Create structured transcript data"""
```

API Response Models

Add these Pydantic models:

```
class TranscriptData(BaseModel):
    user_input: str
    agent_response: str

class EnhancedAudioResponse(BaseModel):
    success: bool
    audio_response: str # base64 encoded
    transcript: TranscriptData
    processing_time_ms: int

class EnhancedTextResponse(BaseModel):
    response_text: str
    processing_time_ms: int
```

Testing Requirements

New Test Cases

You must verify:

1. **Transcript Accuracy:** User input matches STT output
2. **Response Consistency:** Audio and text responses are identical
3. **Timing Data:** Processing time is captured correctly
4. **UI Functionality:** Enhanced chat tab works correctly

Testing Commands

```
# Test modified audio endpoint (now returns JSON with transcript)
curl -X POST http://localhost:8000/chat/audio \
  -F "audio=@test.wav" | jq .

# Test text endpoint (unchanged)
curl -X POST http://localhost:8000/chat/text \
  -H "Content-Type: application/json" \
  -d '{"text": "What is your return policy?"}' | jq .
```

Evaluation Criteria Update

Mid-Session Evaluation (100%)

Enhanced Functionality (85%)

- **Transcript Generation (30%):** Accurate capture of all text interactions
- **Processing Time Collection (20%):** Proper timing measurement
- **API Modification (20%):** Modified `/chat/audio` endpoint works correctly
- **UI Enhancement (15%):** Modified Audio Chat tab displays transcripts

Integration Quality (15%)

- **Consistency (10%):** Audio and text responses match
- **Error Handling (5%):** Graceful failure when components fail

Implementation Strategy

Phase 1: Pipeline Enhancement

1. Add transcript capture to existing pipeline methods
2. Implement timing measurement for each stage
3. Create data structures for transcript storage

Phase 2: API Updates

1. Modify `/chat/audio` endpoint to return JSON instead of raw audio bytes
2. Update response models to include transcript data
3. Test modified endpoint thoroughly

Phase 3: UI Enhancement

1. Modify existing Audio Chat tab in Streamlit app
2. Update to handle JSON response format from `/chat/audio`
3. Add transcript display alongside audio controls
4. Add processing time display

Success Criteria

Your enhanced system is successful when:

Modified `/chat/audio` endpoint returns JSON with audio + transcript data

Processing time is captured and returned for all operations

Modified Streamlit Audio Chat tab displays transcripts alongside audio controls

Audio can still be played back from the base64-encoded response

All other existing functionality continues to work unchanged

Pro Tips

1. **Start with Pipeline:** Get transcript capture working first
2. **Test Incrementally:** Verify each component before moving to the next
3. **Preserve Existing APIs:** Don't break current functionality
4. **Use the Health Endpoint:** Monitor what's working during development
5. **JSON Formatting:** Use proper JSON structure for transcript data

Bonus Challenges

- **Better Audio Format:** Support multiple audio formats (MP3, WAV, M4A)
- **Real-time Processing:** Show progress indicators during processing
- **Error Recovery:** Graceful handling when transcript generation fails
- **Audio Visualization:** Add waveform display for recorded audio

Good luck with your enhanced implementation! This mid-session requirement will significantly improve your system's usability and debuggability.